Floci & Kubernetes PowerShell Reference Guide

Complete step-by-step PowerShell command manual for local AWS emulation & Kubernetes cluster practice

1 ENVIRONMENT VARIABLES SETUP (POWERSHELL)

Configure local environment variables so AWS CLI redirects traffic to Floci (`localhost:4566`) with dummy credentials:

```
# Set AWS endpoints and test credentials for current PowerShell session
$env:AWS_ENDPOINT_URL = "http://localhost:4566"
$env:AWS_DEFAULT_REGION = "us-east-1"
$env:AWS_ACCESS_KEY_ID = "test"
$env:AWS_SECRET_ACCESS_KEY = "test"

# Verify configuration
Get-ChildItem env:AWS_*
```

2 START FLOCI LOCAL EMULATOR (DOCKER COMPOSE)

Create `compose.yaml` in your project directory (e.g., `C:\Krishna\Practice\kubern8`):

```
# compose.yaml
services:
  floci:
    image: floci/floci:latest
    container_name: floci
    ports:
      - "4566:4566"
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"
    environment:
      - DOCKER_HOST=unix:///var/run/docker.sock
```

Start and check the Floci container in PowerShell:

```
# Start Floci container
docker compose up -d

# Verify container status
docker ps
```

3 CREATE LOCAL EKS KUBERNETES CLUSTER

Use AWS CLI to request an EKS cluster from Floci. Pass dummy IAM Role and VPC configs to satisfy local AWS CLI validation:

```
# Create EKS cluster with required dummy parameters
aws eks create-cluster --name dev-cluster --role-arn arn:aws:iam::123456789012:role/dummy-role
--resources-vpc-config subnetIds=subnet-12345678,securityGroupIds=sg-12345678

# Verify cluster status
aws eks describe-cluster --name dev-cluster
```

4 CONFIGURE KUBECTL CREDENTIALS

Update your local `kubeconfig` to direct `kubectl` commands to the Floci cluster:

```
# Generate kubeconfig entry for dev-cluster
aws eks update-kubeconfig --name dev-cluster

# Verify cluster nodes
kubectl get nodes -o wide
```

5 KUBERNETES PRACTICE WORKFLOW

Standard `kubectl` deployment and cluster management commands:

```
# Deploy Nginx application
kubectl create deployment nginx-demo --image=nginx

# Expose deployment via NodePort
kubectl expose deployment nginx-demo --port=80 --type=NodePort

# View running Pods, Deployments, and Services
kubectl get pods
kubectl get deployments
kubectl get svc

# Inspect Pod logs and details
kubectl logs -l app=nginx-demo
kubectl describe pod -l app=nginx-demo
```

6 STANDALONE EC2 / MANUAL K3S SETUP (ALTERNATIVE)

If creating an EC2 container directly instead of EKS, run K3s manually inside the container:

```
# 1. Launch EC2 container
aws ec2 run-instances --image-id ami-12345678 --count 1 --instance-type t3.micro --key-name my-
key

# 2. Exec into container
docker exec -it <container_id> sh

# 3. Inside container shell: Download and run K3s directly (without systemd)
curl -Lo /usr/local/bin/k3s https://github.com/k3s-io/k3s/releases/download/v1.28.2%2Bk3s1/k3s
chmod +x /usr/local/bin/k3s
k3s server --write-kubeconfig-mode 644 &
k3s kubectl get nodes
exit
```

7 CLEANUP COMMANDS

Tear down local Kubernetes and AWS emulator resources when finished:

```
# Delete Kubernetes deployment and service
kubectl delete service nginx-demo
kubectl delete deployment nginx-demo

# Delete EKS cluster from Floci
aws eks delete-cluster --name dev-cluster

# Stop and remove Floci docker containers
docker compose down
```